SENTINEL Lab 1 Guide

Your watch, and the ideas behind `libsyscall`

ECE 476 Engineering System Software • Fall 2026 • University of Victoria

This guide starts with the watch that Lab 1 is for, and then it explains the ideas behind Lab 1. It states no rule. The Lab 1 Brief, `docs/lab1/lab1_brief.pdf`, states each rule. The repository states each technical detail: `syscall.h`[1] for each function of the library, and `common/linereader.h` for the reader of the feed.

1 Your watch

You are a software engineer at a surveillance detachment of the Canadian Armed Forces. The detachment covers the northern and Pacific approaches of Canada. It operates four sensors and one operations centre, as Figure 1 shows. A watch officer sits at the display of the centre for the length of a watch, and acts on what the display shows. The course Lab Documentation[2] describes the detachment and each sensor.

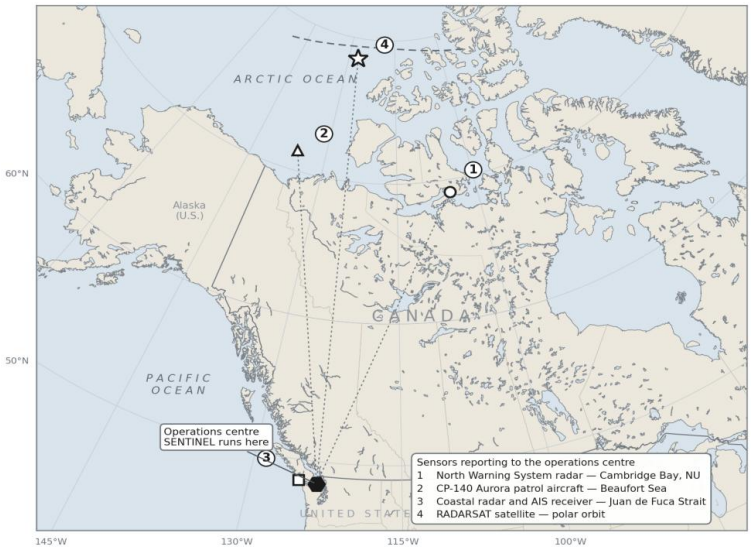


Figure 1. The four sensors of the detachment, and the operations centre that they report to. Positions are approximate. The figure is from the ECE 476 Lab Documentation.

No sensor is enough alone. The radar watches all the time, but it cannot tell what a contact is. The Aurora can identify a contact, but it can be in one place only. AIS names the ships that broadcast, so a radar contact with no AIS is a contact that a watch officer wants to know about.

The satellite covers the whole area. The areas overlap, so one object can be in the reports of two sensors, each with a slightly different position. The software that combines the reports makes the four sensors useful.

[1] `src/libsyscall/include/sentinel/syscall.h`, in your repository. It states what each function of Lab 1 does, and each of its errors.

[2] The ECE 476 Lab Documentation for SENTINEL, on Brightspace. It describes the detachment, the four sensors, the four parts of SENTINEL, and a user story.

1.1 The rule of the system

Machines watch. People decide.

This is the rule in Canadian and NORAD practice. Software collects the reports, records them, and sorts them by importance. A person decides what to do about them. To send fighters from Cold Lake or Bagotville to look at an unidentified aircraft is always the decision of a person.

So your software has one duty above all others: **give the watch officer a true account, in time to act on it.** Nobody can act on a report that the centre did not read. Nobody can review a report that the centre did not record. Nobody can put a report with no correct time in order with other events.

1.2 The current watch

Watch serial	**2026-282**
Detachment	SENTINEL, northern and Pacific approaches
Sensors	Four, and each one reports for the whole watch
Centre	It records each report. It cannot yet decide what a contact is
Open item	A contact in the strait broadcasts an identity that is on no filed list

A vessel in the Juan de Fuca Strait broadcasts an identity that is on no filed list of the detachment. It can be a registration with an error, a transmitter that nobody registered, or a vessel that does not want to be known. The centre cannot decide which, and Lab 1 does not give it the means. Lab 1 gives the centre the record: each report about the vessel, with a correct time, and nothing lost. A later lab, Lab 3,[3] compares each report with the filed traffic and raises the alert, but it can do that only with a complete record. That record is your work in Lab 1.

1.3 Why Lab 1 is first

The centre cannot decide anything about a report that it did not read. It cannot prove anything that it did not record. It cannot put in order anything that it cannot timestamp. These three sentences are Lab 1. Each later lab reads its input and writes its history through your library, so no later lab can be correct if this layer is wrong.

2 Three operational cases

Your library takes part in each of these three operations. Each case names the functions that do the work. The Lab 1 Brief[4] states their rules.

Case 1. Normal operations

1. The watch officer starts the operations centre at the start of a watch.
2. The four sensors report: the radar about 20 reports each second, the Aurora about 5, the coastal receiver about 50, and the satellite about 2000 in each second of a pass.

[3] SENTINEL is built in four labs, and each lab adds one library. The SENTINEL Lab Manual, Section 1.1, `docs/manual/sentinel_manual.pdf`, states what each lab adds.

[4] The Lab 1 Brief, `docs/lab1/lab1_brief.pdf` in your repository. It states the contract of Lab 1.

3. The centre reads each report, gives it a time, and writes it to the mission log.
4. The display shows a routine watch: reports arrive, and nothing is open.

In the code: `sc_open_read` opens the feed, and `lr_next_line` gives each line. `contact_parse` makes a record from the line, and `log_writef` writes it to the mission log with its time.

`make lab1`[5] replays a recorded feed: the sensors write a file first, and then the demo[6] reads it to the end. A detachment replays a recorded feed to answer a question about a watch that ended, as in Case 3. A live watch reads the feed while the sensors still write it (Section 5).

Case 2. A contact that is on no list

1. The coastal receiver in the Juan de Fuca Strait reports a vessel. The vessel broadcasts an identity, as each vessel in the lane does.
2. That identity is on no list of the detachment. Each other vessel in the lane matches a filed registration.
3. Your library reads the report while routine traffic arrives from the other three sensors. It gives the report a time, and it writes it to the mission log with the other reports.
4. RADARSAT can report the same vessel from a slightly different position. The centre records both reports.

In the code: the same functions as in Case 1. Lab 1 does not treat the vessel in a different way, and it must not.

What Lab 1 does not do yet. It does not compare the identity with the filed traffic, so it raises no alert. It also does not know that two reports from two sensors are one vessel. Lab 3 decides both.

What Lab 1 decides is if anyone can answer the question at all. An alert in Lab 3 compares a record with a list. Lab 1 can drop the reports, give them a wrong time, or stop the read of the feed after one bad line. Then the vessel was never there for the centre. No later lab can recover it.

See it: run `make lab1`, then `make watch`,[7] and look at the Mission log view. The identity of each vessel in the strait starts with `VESL`. One of them is the vessel of this case, and nothing on the screen marks it. That is correct for Lab 1.

Case 3. Someone questions the record

1. After the watch, someone asks the detachment what the centre knew about the vessel of Case 2, and when it knew it.
2. Nobody from the watch is there, and no screen still shows that day.
3. The mission log alone gives the answer.

[5] `make lab1` writes a new feed, `build/live/feed.txt`, from the four sensors, and it runs your demo on it. The SENTINEL Lab Manual, Section 4, `docs/manual/sentinel_manual.pdf`, lists each make target.

[6] `demo_lab1` is the demonstration program of Lab 1. It reads a feed, a file of contact reports, and writes the mission log. The Lab 1 Brief, Section 3, `docs/lab1/lab1_brief.pdf`, specifies it.

[7] The page is the operations centre display. Start it with `make watch`, and open `http://localhost:8080/`. `web/README.md`, in your repository, describes the page and each view.

This case gives three decisions:

- The timestamp is a date from `CLOCK_REALTIME`, not a count of seconds from the start of the program.
- The mission log uses `O_APPEND`, because from Lab 2 several processes write it, and a record with lost lines cannot answer a question about them.
- The mission log has the mode `0600`, because it is a surveillance record.

In the code: `log_open`, `log_writef` and `log_close`, with the time of `sc_stamp`.

3 Lab 1 in SENTINEL

Lab 1 builds `libsyscall`, the layer of SENTINEL that reads the feed, gives the time, and writes the mission log. In Lab 1, one program, `demo_lab1`,[8] reads a file of contact reports that `contactfeed`[9] writes, as Figure 2 shows. `make watch` shows the result on the page.[10]From Lab 2, four sensor processes send the reports.

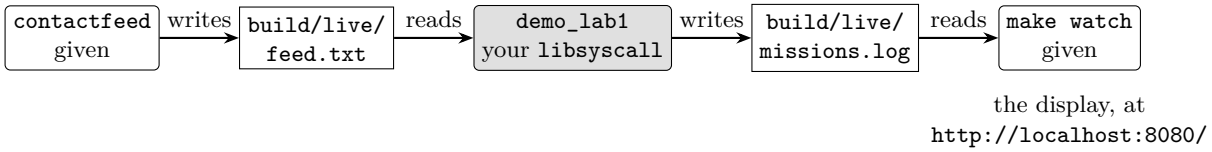


Figure 2. The data flow of Lab 1. The grey box is your code. A box with square corners is a file.

Lab 1 satisfies requirement R6.[11] Each action has a timestamp, and the centre writes it to the mission log. A mission log with a gap cannot answer a question about the gap. A mission log with a wrong clock cannot put two events in order.

Each lab adds one thing that the centre can do. The table shows what the centre can do after Lab 1, and what a later lab adds.

The centre can	From	A later lab adds
Read a feed, and lose none of it	Lab 1	
Put a real date on each report	Lab 1	
Record each report, in order	Lab 1	
	Lab 2	Run each sensor in its own process, and continue when one stops
	Lab 2	Let an urgent report overtake routine traffic
	Lab 3	Take thousands of reports each second
	Lab 3	Decide that two reports are one object, and compare it with the filed traffic
	Lab 4	Keep that picture exact while many workers change it

[8] `demo_lab1` is the demonstration program of Lab 1. It reads a feed, a file of contact reports, and writes the mission log. The Lab 1 Brief, Section 3, `docs/lab1/lab1_brief.pdf`, specifies it.

[9] `contactfeed` is the given program that writes contact reports in place of the four sensors. The SENTINEL Lab Manual, Section 3.1, `docs/manual/sentinel_manual.pdf`, describes it.

[10] The page is the operations centre display. Start it with `make watch`, and open `http://localhost:8080/`. `web/README.md`, in your repository, describes the page and each view.

[11] R6 is a requirement of SENTINEL: each action has a timestamp, and the centre writes it to the mission log. The SENTINEL Lab Manual, Section 1.2, `docs/manual/sentinel_manual.pdf`, lists the six requirements.

So the vessel in the strait stays open at the end of Lab 1, and that is the correct result. A centre that cannot yet identify a contact must still record it exactly.

The table shows where SENTINEL uses each function: in Lab 1, and in the labs after it.

Function	In Lab 1[12]	After Lab 1
`sc_open_read`	Step 4: the demo opens the feed	Lab 4: the complete program reads its mission log again after a run
`sc_open_write`	not in the demo	No program. Only the tests use it
`sc_open_write_mode`	not in the demo	No program outside the library
`sc_close`	Step 5: the demo closes the feed after an error	Lab 2: the centre closes the priority FIFO. Lab 4: the complete program closes its mission log after an error
`sc_write_all`	not in the demo	Lab 2: one line to the priority FIFO, and the statistics of the centre
`sc_read_full`	not in the demo	No program. Only the tests use it
`sc_now_ns`	Steps 6 and 8: the elapsed time	Labs 2, 3 and 4: each duration, for example the delay of a report
`sc_now_wall`	not in the demo	Lab 3: the centre compares the time of a report with the real time, to find a replay
`sc_stamp`	not in the demo	Lab 4: the complete program puts the time on an urgent report that it makes
`log_open`	Step 1	Labs 2, 3 and 4: each demonstration program, and the complete program, opens a mission log
`log_writef`	Each line of the mission log	Labs 2, 3 and 4: each event of the sensors, the centre and the workers
`log_close`	Step 11	Labs 2, 3 and 4: each demonstration program, and the complete program
`log_level_name`	not in the demo	Lab 4: the complete program finds each level name in its mission log
`proc_report`	Steps 3 and 9	No program after Lab 1
`proc_report_write`	Step 10	No program after Lab 1

Your library can also call its own functions. For example, `log_writef` can call `sc_write_all`.

4 The ideas behind Lab 1

4.1 A system call can do less than you ask

`read` can return fewer bytes than you ask for, and `write` can take fewer bytes than you give it. A signal can stop a call that waits, and the call then fails with `EINTR`. Neither result is a fault, so each wrapper of Lab 1 calls the system call again until the work is done. A program that expects one call to move all the bytes works on a small file. On a pipe or a terminal, it loses data. `syscall.h`[13] states what each wrapper does.

[12] A step is one of the twelve steps of `demo_lab1`, the demonstration program of Lab 1, in order. The Lab 1 Brief, Section 3.2, `docs/lab1/lab1_brief.pdf`, lists them.

[13] `src/libsyscall/include/sentinel/syscall.h`, in your repository. It states what each function of Lab 1 does, and each of its errors.

4.2 Two clocks

`CLOCK_REALTIME` gives the date. A time service or an administrator can move it forward or back while a program runs. `CLOCK_MONOTONIC` never moves back, so a duration uses it, but its value is not a date. Figure 3 shows one duration, read from each clock.

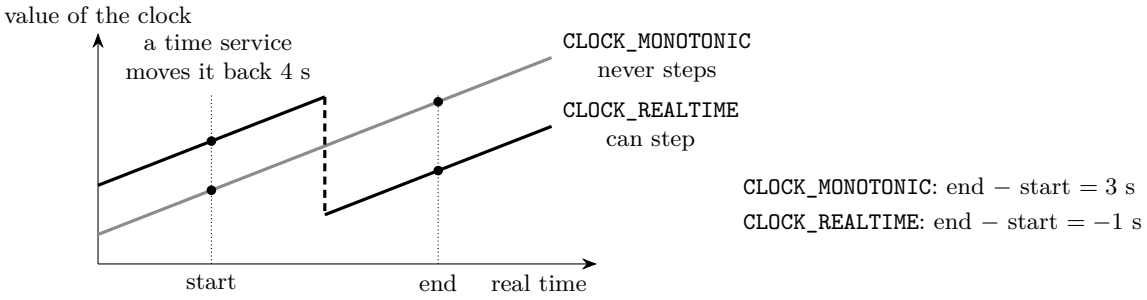


Figure 3. One duration, read from each clock. Only `CLOCK_MONOTONIC` gives the correct duration. Its value is not a date: only the difference of two values is a duration.

4.3 One mission log, many writers

From Lab 2, four sensor processes and the centre write the same mission log. With `O_APPEND`, each `write` goes to the end of the file in one step. Without it, each process keeps its own offset, and one line goes over another with no error, as Figure 4 shows. A mission log that loses lines with no error is worse than no mission log, because people believe it.

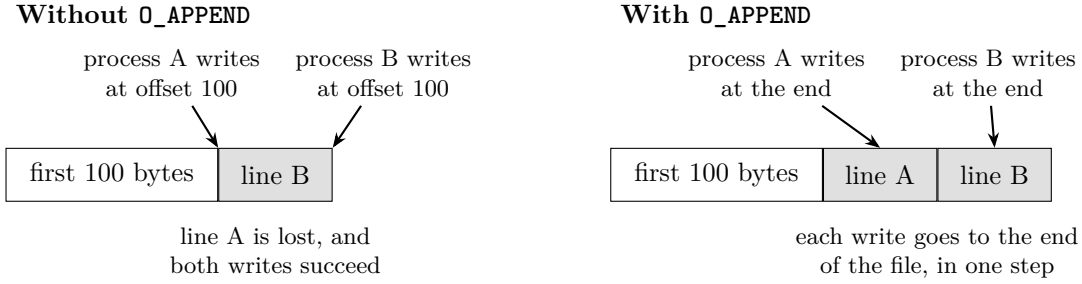


Figure 4. Two processes write the same mission log. Without `O_APPEND`, each process keeps its own offset, and the second line goes over the first.

4.4 A file that still grows

On a regular file, `read` returns 0 when there is nothing after the offset *yet*. A different program can still add lines, and the next `read` returns them. `tail -f` uses this fact. So the reader from `lr_open_follow` reports `EAGAIN`, "nothing yet", in place of the end of the feed. `EMSGSIZE` is different: one line was too long and is lost, but the next line still arrives. `common/linereader.h` states each result, and the Lab 1 Brief shows the read loop.[14]

4.5 The kernel's view of a process

Linux describes each process that runs in the directory `/proc/<pid>`. Your `proc_report` reads it: the name, the parent, the memory, the threads, the open descriptors and the context switches.

[14]The read loop of `demo_lab1`: the Lab 1 Brief, Section 3.4 and its Figure 1, `docs/lab1/lab1_brief.pdf`.

A program that cannot report its own state cannot show that it leaks memory or descriptors. `syscall.h`[15] states each field and the file that it comes from.

5 See it run

A live watch shows the read loop of the demo[16] at work. It uses three terminals, at the top of your repository. Terminal 1 runs `contactfeed`[17] with the profile `world`,[18] and Terminal 3 starts the page.[19]Start Terminal 1 first, because it creates the feed file.

```
# Terminal 1: three sensors, each at its own rate
mkdir -p build/live
rm -f build/live/feed.txt build/live/missions.log
touch build/live/feed.txt
build/contactfeed --sensor NWS --rate 20 --profile world >> build/live/feed.txt &
build/contactfeed --sensor AUR --rate 5  --profile world >> build/live/feed.txt &
build/contactfeed --sensor AIS --rate 50 --profile world >> build/live/feed.txt &
wait

# Terminal 2: your demo reads while the three sensors write
build/demo_lab1 build/live/feed.txt build/live/missions.log --follow 3000

# Terminal 3: the page
make watch
```

The two `rm` commands remove the feed and the mission log of an earlier run. Without them, the demo adds the new run to the old mission log. Of these three sensors, only AIS reports the vessels in the strait. Open `http://localhost:8080/` and select the Mission log view: it fills while the run continues. To stop, press Ctrl-C in Terminal 1. The demo waits three seconds for a new line, then it prints its summary and stops.

[15] `src/libsyscall/include/sentinel/syscall.h`, in your repository. It states what each function of Lab 1 does, and each of its errors.

[16] `demo_lab1` is the demonstration program of Lab 1. It reads a feed, a file of contact reports, and writes the mission log. The Lab 1 Brief, Section 3, `docs/lab1/lab1_brief.pdf`, specifies it.

[17] `contactfeed` is the given program that writes contact reports in place of the four sensors. The SENTINEL Lab Manual, Section 3.1, `docs/manual/sentinel_manual.pdf`, describes it.

[18] A profile selects the traffic that `contactfeed` sends: `world`, `ideal`, `dirty`, `field` or `threat`. The SENTINEL Lab Manual, Section 3.1, `docs/manual/sentinel_manual.pdf`, describes each one.

[19] The page is the operations centre display. Start it with `make watch`, and open `http://localhost:8080/`. `web/README.md`, in your repository, describes the page and each view.

What you see	What it means
`lines read 0`, and the demo[20] stops after three seconds	Terminal 1 did not run, so the feed file is empty
The demo cannot open the feed	`build/live/feed.txt` does not exist. Run the `touch` command in Terminal 1
The demo stops at once, and the count is near 1000	The feed still holds the run of `make lab1`.[21] Run the two `rm` commands
The demo does not stop	The read loop has no idle limit

While the demo runs, look at its entry in `/proc` from a fourth terminal:

```
pid=$(pgrep -nx demo_lab1)
watch -n 1 "grep -E '^(State|voluntary|nonvoluntary)' /proc/$pid/status; \
  head -n 2 /proc/$pid/fdinfo/3"
```

Look for three things:

- `State:` shows `S (sleeping)` for most of the run. The demo waits, and it does not use the processor. A loop with no wait shows `R (running)`.
- `voluntary_ctxt_switches` increases by about 20 each second: each wait of 50 ms gives the processor up. With no wait, `nonvoluntary_ctxt_switches` increases in its place, because the kernel must take the processor away.
- `flags:` of descriptor 3, the mission log, ends in `2001`: `2000` is `O_APPEND`, and `1` is `O_WRONLY`. The full value is `0402001` on aarch64 and `0102001` on x86_64.

When the demo stops, `/proc/<pid>` no longer exists, because the kernel removes the directory with the process.

[20] `demo_lab1` is the demonstration program of Lab 1. It reads a feed, a file of contact reports, and writes the mission log. The Lab 1 Brief, Section 3, `docs/lab1/lab1_brief.pdf`, specifies it.

[21] `make lab1` writes a new feed, `build/live/feed.txt`, from the four sensors, and it runs your demo on it. The SENTINEL Lab Manual, Section 4, `docs/manual/sentinel_manual.pdf`, lists each make target.